

ЛАБОРАТОРНА РОБОТА № 4

Тема: Класифікація тональності тексту (Negative / Neutral / Positive) за допомогою LSTM (PyTorch)

Хід роботи:

Завдання 1: Обрати англomовний датасет речень з трьома класами тональності (negative, neutral, positive).

Завдання 2: Завантажити датасет через бібліотеку Hugging Face datasets (twitter-airline-sentiment, fallback: tweet_eval/sentiment).

Завдання 3: Підготувати текстові дані: токенизація, побудова словника, кодування та padding до сталої довжини.

Завдання 4: Розділити дані на тренувальну, валідаційну та тестову вибірки зі стратифікацією.

Завдання 5: Реалізувати модель LSTM-класифікатора у PyTorch (Embedding + BiLSTM + повнозв'язні шари).

Завдання 6: Додати регуляризацію та стабілізацію навчання (Dropout, gradient clipping, scheduler, early stopping).

Завдання 7: Навчити модель, зберегти найкращий стан за метрикою val_macro_f1.

Завдання 8: Оцінити якість на тестовій вибірці: accuracy, precision/recall/f1, confusion matrix.

Завдання 9: Протестувати модель на нових реченнях та зберегти візуалізацію результатів у lab4_lstm_results.png.

Лістинг програми:

```
import matplotlib
import numpy as np

matplotlib.use('Agg')
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from matplotlib.gridspec import GridSpec

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from torch.nn.utils.rnn import pack_padded_sequence
from collections import Counter
from datasets import load_dataset
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, f1_score
import warnings
warnings.filterwarnings('ignore')
```

					ДУ «Житомирська політехніка».26.121.20.000 – Лр4			
Змн.	Арк.	№ докум.	Підпис	Дата	Звіт з лабораторної роботи ФІКТ Гр. ІПЗ-23-1			
Розроб.		Рижченко Я.В.						
Перевір.		Годлевський Ю.О						
Керівник								
Н. контр.								
Зав. каф.								
					Лім.	Арк.	Аркушів	
						1	8	

```

SEED = 42
torch.manual_seed(SEED)
np.random.seed(SEED)

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {DEVICE}")

def load_sentiment_texts_and_labels() -> tuple[list[str], list[int], str]:
    try:
        ds = load_dataset("osanseviero/twitter-airline-sentiment", split="train")
        label_map = {"negative": 0, "neutral": 1, "positive": 2}
        texts = [str(t) for t in ds["text"]]
        labels = [label_map[str(lbl).lower()] for lbl in ds["airline_sentiment"]]
        return texts, labels, "osanseviero/twitter-airline-sentiment"
    except Exception:
        ds = load_dataset("tweet_eval", "sentiment", split="train")
        texts = [str(t) for t in ds["text"]]
        labels = [int(lbl) for lbl in ds["label"]] # 0=negative, 1=neutral, 2=positive
        return texts, labels, "tweet_eval/sentiment"

texts, labels, dataset_name = load_sentiment_texts_and_labels()

MAX_SAMPLES = 14000
if len(texts) > MAX_SAMPLES:
    X_tmp, _, y_tmp, _ = train_test_split(
        texts, labels, train_size=MAX_SAMPLES, random_state=SEED, stratify=labels
    )
    texts, labels = X_tmp, y_tmp

print(f"Dataset: {dataset_name}")
print(f"Samples: {len(texts)} "
      f"(positive={labels.count(2)}, neutral={labels.count(1)}, negative={labels.count(0)})")

MAX_WORDS = 5000
MAX_LEN = 40

def simple_tokenize(text: str) -> list[str]:
    import re
    return re.findall(r"[a-z]+'", text.lower())

counter = Counter(w for t in texts for w in simple_tokenize(t))
vocab = ['<PAD>', '<OOV>'] + [w for w, _ in counter.most_common(MAX_WORDS - 2)]
word2idx = {w: i for i, w in enumerate(vocab)}
VOCAB_SIZE = len(vocab)

def encode(text: str) -> list[int]:
    return [word2idx.get(w, 1) for w in simple_tokenize(text)]

def pad_encode(text: str, maxlen: int = MAX_LEN) -> list[int]:
    ids = encode(text)[:maxlen]
    return ids + [0] * (maxlen - len(ids))

sequences = [encode(t) for t in texts]
padded = np.array([pad_encode(t) for t in texts], dtype=np.int64)

print(f"\nVocabulary size : {VOCAB_SIZE} words")
print(f"Example sentence: {texts[0]}")
print(f"Token sequence : {sequences[0]}")
print(f"After padding : {padded[0]}")

X_train_np, X_test_np, y_train_np, y_test_np = train_test_split(
    padded, np.array(labels, dtype=np.int64),
    test_size=0.2, random_state=SEED, stratify=labels
)
print(f"\nTraining samples: {len(X_train_np)} | Test samples: {len(X_test_np)}")

class SentimentDataset(Dataset):
    def __init__(self, X: np.ndarray, y: np.ndarray):
        self.X = torch.tensor(X, dtype=torch.long)

```

		Риженко Я.В			ДУ «Житомирська політехніка».26.121.20.000 – Пр4	Арк.
		Годлевський Ю.О				
Змн.	Арк.	№ докум.	Підпис	Дата		2

```

self.y = torch.tensor(y, dtype=torch.long)
self.lengths = torch.tensor((X != 0).sum(axis=1), dtype=torch.long)

def __len__(self):
    return len(self.y)

def __getitem__(self, idx):
    return self.X[idx], self.y[idx], self.lengths[idx]

test_ds = SentimentDataset(X_test_np, y_test_np)
test_loader = DataLoader(test_ds, batch_size=32, shuffle=False)

class LSTMClassifier(nn.Module):
    def __init__(self, vocab_size: int, embed_dim: int = 128,
                  hidden: int = 96, num_classes: int = 3, dropout: float = 0.35):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.lstm = nn.LSTM(
            embed_dim, hidden, num_layers=2, batch_first=True,
            bidirectional=True, dropout=dropout
        )
        self.dropout = nn.Dropout(dropout)
        self.fc1 = nn.Linear(hidden * 2, 64)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(64, num_classes)

    def forward(self, x: torch.Tensor, lengths: torch.Tensor) -> torch.Tensor:
        emb = self.embedding(x)
        packed = pack_padded_sequence(
            emb, lengths.cpu(), batch_first=True, enforce_sorted=False
        )
        _, (hn, _) = self.lstm(packed)
        out = torch.cat((hn[-2], hn[-1]), dim=1)
        out = self.dropout(out)
        out = self.relu(self.fc1(out))
        return self.fc2(out)

model = LSTMClassifier(vocab_size=VOCAB_SIZE).to(DEVICE)
print(f"\nModel architecture:\n{model}")
total_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"Trainable parameters: {total_params:,}")

X_tr, X_val, y_tr, y_val = train_test_split(
    X_train_np, y_train_np, test_size=0.15, random_state=SEED, stratify=y_train_np
)
tr_ds = SentimentDataset(X_tr, y_tr)
val_ds = SentimentDataset(X_val, y_val)
val_loader = DataLoader(val_ds, batch_size=32, shuffle=False)

class_counts = np.bincount(y_tr, minlength=3).astype(np.float32)
class_weights = class_counts.sum() / (len(class_counts) * class_counts)
class_weights_t = torch.tensor(class_weights, dtype=torch.float32, device=DEVICE)
print(f"Class weights (train split): {class_weights.round(3).tolist()}")

criterion = nn.CrossEntropyLoss(weight=class_weights_t, label_smoothing=0.05)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-5)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode='min', factor=0.5, patience=2, min_lr=1e-5
)

EPOCHS = 35
PATIENCE = 8
MIN_EPOCHS = 10
tr_loader = DataLoader(tr_ds, batch_size=32, shuffle=True)

history = {'loss': [], 'accuracy': [], 'val_loss': [], 'val_accuracy': [], 'val_macro_f1': []}
best_state = None
best_val_f1 = -1.0
epochs_without_improve = 0

```

		Риженко Я.В			ДУ «Житомирська політехніка».26.121.20.000 – Пр4	Арк.
		Годлевський Ю.О				
Змн.	Арк.	№ докум.	Підпис	Дата		3

```

for epoch in range(1, EPOCHS + 1):
    model.train()
    run_loss, correct, total = 0.0, 0, 0
    for xb, yb, lb in tr_loader:
        xb, yb, lb = xb.to(DEVICE), yb.to(DEVICE), lb.to(DEVICE)
        optimizer.zero_grad()
        logits = model(xb, lb)
        loss = criterion(logits, yb)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        optimizer.step()
        run_loss += loss.item() * len(yb)
        correct += (logits.argmax(1) == yb).sum().item()
        total += len(yb)

    epoch_loss = run_loss / total
    epoch_acc = correct / total

    model.eval()
    val_loss, val_correct, val_total = 0.0, 0, 0
    val_preds_epoch, val_labels_epoch = [], []
    with torch.no_grad():
        for xb, yb, lb in val_loader:
            xb, yb, lb = xb.to(DEVICE), yb.to(DEVICE), lb.to(DEVICE)
            logits = model(xb, lb)
            loss = criterion(logits, yb)
            preds = logits.argmax(1)
            val_loss += loss.item() * len(yb)
            val_correct += (preds == yb).sum().item()
            val_total += len(yb)
            val_preds_epoch.extend(preds.cpu().numpy())
            val_labels_epoch.extend(yb.cpu().numpy())

    val_epoch_loss = val_loss / val_total if val_total else 0.0
    val_epoch_acc = val_correct / val_total if val_total else 0.0
    val_macro_f1 = f1_score(val_labels_epoch, val_preds_epoch, average='macro')

    history['loss'].append(epoch_loss)
    history['accuracy'].append(epoch_acc)
    history['val_loss'].append(val_epoch_loss)
    history['val_accuracy'].append(val_epoch_acc)
    history['val_macro_f1'].append(val_macro_f1)
    scheduler.step(val_epoch_loss)

    if val_macro_f1 > best_val_f1 + 1e-4:
        best_val_f1 = val_macro_f1
        best_state = {k: v.detach().cpu().clone() for k, v in model.state_dict().items()}
        epochs_without_improve = 0
    else:
        epochs_without_improve += 1

    if epoch % 5 == 0 or epoch == 1:
        print(f'Epoch {epoch:>3}/{EPOCHS} '
              f'loss={epoch_loss:.4f} acc={epoch_acc:.3f} '
              f'val_loss={val_epoch_loss:.4f} val_acc={val_epoch_acc:.3f} '
              f'val_f1={val_macro_f1:.3f}')
    if epoch >= MIN_EPOCHS and epochs_without_improve >= PATIENCE:
        print(f'Early stopping at epoch {epoch} (best val_macro_f1={best_val_f1:.3f})')
        break

    if best_state is not None:
        model.load_state_dict(best_state)

    model.eval()
    all_preds, all_labels = [], []
    test_loss_total, test_correct, test_total = 0.0, 0, 0

    with torch.no_grad():
        for xb, yb, lb in test_loader:
            xb, yb, lb = xb.to(DEVICE), yb.to(DEVICE), lb.to(DEVICE)
            logits = model(xb, lb)
            loss = criterion(logits, yb)

```

		Риженко Я.В			ДУ «Житомирська політехніка».26.121.20.000 – Пр4	Арк.
		Годлевський Ю.О				
Змн.	Арк.	№ докум.	Підпис	Дата		4

```

preds = logits.argmax(1)
test_loss_total += loss.item() * len(yb)
test_correct += (preds == yb).sum().item()
test_total += len(yb)
all_preds.extend(preds.cpu().numpy())
all_labels.extend(yb.cpu().numpy())

test_acc = test_correct / test_total
test_loss = test_loss_total / test_total
print(f"\nTest accuracy: {test_acc * 100:.1f}% | Loss: {test_loss:.4f}")

class_names = ['Negative', 'Neutral', 'Positive']
print("\nDetailed classification report:\n")
print(classification_report(all_labels, all_preds, target_names=class_names))

cm = confusion_matrix(all_labels, all_preds)

def predict_sentiment(text: str):
    model.eval()
    ids = torch.tensor([pad_encode(text)], dtype=torch.long).to(DEVICE)
    lens = torch.tensor([max(1, len(encode(text)))]).to(DEVICE)
    with torch.no_grad():
        logits = model(ids, lens)
        probs = torch.softmax(logits, dim=-1).squeeze().cpu().numpy()
    cls = int(np.argmax(probs))
    return class_names[cls], probs

demo = [
    "I love this, it is truly wonderful!",
    "The item was delivered on Wednesday.",
    "This is the worst thing I have ever bought.",
    "Absolutely fantastic results, very happy!",
    "The document has been processed.",
    "Terrible quality, I want my money back.",
]

print("\n-- Sentiment predictions on new sentences --")
demo_results = []
for t in demo:
    label, probs = predict_sentiment(t)
    demo_results.append((t, label, probs))
    print(f" [{label:>8}] {t}")

COLORS = {'Negative': '#e74c3c', 'Neutral': '#3498db', 'Positive': '#2ecc71'}

fig = plt.figure(figsize=(18, 14))
fig.patch.set_facecolor('#0f1117')
gs = GridSpec(3, 3, figure=fig, hspace=0.45, wspace=0.35)

ax_loss = fig.add_subplot(gs[0, 0])
ax_acc = fig.add_subplot(gs[0, 1])
ax_cm = fig.add_subplot(gs[0, 2])
ax_bar = fig.add_subplot(gs[1, :])
ax_demo = fig.add_subplot(gs[2, :])

def style_axes(ax, title):
    ax.set_facecolor('#1a1d2e')
    ax.tick_params(colors='#cdd6f4', labelsz=9)
    for sp in ax.spines.values():
        sp.set_color('#313244')
    ax.set_title(title, color='#cdd6f4', fontsize=11, pad=8)

style_axes(ax_loss, 'Loss Curve')
ax_loss.plot(history['loss'], color='#f38ba8', lw=2, label='Train')
ax_loss.plot(history['val_loss'], color='#fab387', lw=2, label='Val', linestyle='--')
ax_loss.set_xlabel('Epoch', color='#a6adc8', fontsize=9)
ax_loss.legend(facecolor='#1a1d2e', labelcolor='#cdd6f4', fontsize=9)

style_axes(ax_acc, 'Accuracy Curve')

```

		Риженко Я.В			ДУ «Житомирська політехніка».26.121.20.000 – Пр4	Арк.
		Годлевський Ю.О				
Змн.	Арк.	№ докум.	Підпис	Дата		5

```

ax_acc.plot(history['accuracy'], color='#a6e3a1', lw=2, label='Train')
ax_acc.plot(history['val_accuracy'], color='#89dceb', lw=2, label='Val', linestyle='--')
ax_acc.set_xlabel('Epoch', color='#a6adc8', fontsize=9)
ax_acc.set_ylim(0, 1.05)
ax_acc.legend(facecolor='#1a1d2e', labelcolor='#cdd6f4', fontsize=9)

style_axes(ax_cm, 'Confusion Matrix')
ax_cm.imshow(cm, cmap='Blues')
ax_cm.set_xticks(range(3)); ax_cm.set_yticks(range(3))
ax_cm.set_xticklabels(['Neg', 'Neu', 'Pos'], color='#cdd6f4', fontsize=9)
ax_cm.set_yticklabels(['Neg', 'Neu', 'Pos'], color='#cdd6f4', fontsize=9)
ax_cm.set_xlabel('Predicted', color='#a6adc8', fontsize=9)
ax_cm.set_ylabel('Actual', color='#a6adc8', fontsize=9)
for i in range(3):
    for j in range(3):
        ax_cm.text(j, i, cm[i, j], ha='center', va='center',
                    color='white' if cm[i, j] > cm.max() / 2 else '#cdd6f4',
                    fontsize=13, fontweight='bold')

style_axes(ax_bar, 'Class Distribution in Dataset')
cnt = [labels.count(0), labels.count(1), labels.count(2)]
bars = ax_bar.bar(class_names, cnt,
                  color=[COLORS[c] for c in class_names],
                  width=0.45, edgecolor='#313244', linewidth=1.2)
ax_bar.set_ylabel('Number of sentences', color='#a6adc8', fontsize=9)
for b, v in zip(bars, cnt):
    ax_bar.text(b.get_x() + b.get_width() / 2, v + 0.3, str(v),
                ha='center', color='#cdd6f4', fontsize=11, fontweight='bold')
ax_bar.set_ylim(0, max(cnt) + 5)

style_axes(ax_demo, 'Classification Results on New Sentences')
ax_demo.set_xlim(0, 1)
ax_demo.set_ylim(-0.5, len(demo_results) - 0.5)
ax_demo.axis('off')
ax_demo.set_title('Classification Results on New Sentences',
                  color='#cdd6f4', fontsize=11, pad=8)

for i, (text, label, probs) in enumerate(reversed(demo_results)):
    y_pos = i
    col = COLORS[label]
    ax_demo.fill_betweenx([y_pos - 0.40, y_pos + 0.35], 0.01, 0.99,
                          color='#1e2030', linewidth=0)
    ax_demo.fill_betweenx([y_pos - 0.38, y_pos + 0.33], 0.01, 0.14,
                          color=col, alpha=0.2, linewidth=0)
    ax_demo.text(0.075, y_pos - 0.02, label, va='center', ha='center',
                 color=col, fontsize=8, fontweight='bold')
    ax_demo.text(0.16, y_pos - 0.02, text[:65], va='center',
                 color='#cdd6f4', fontsize=8.5)
    for k in range(3):
        bx = 0.72 + k * 0.09
        bar_h = float(probs[k]) * 0.55
        ax_demo.fill_betweenx([y_pos - 0.30, y_pos - 0.30 + bar_h],
                              bx, bx + 0.07,
                              color=list(COLORS.values())[k], alpha=0.85, linewidth=0)
        ax_demo.text(bx + 0.035, y_pos - 0.42, f'{probs[k] * 100:.0f}%',
                     ha='center', color='#a6adc8', fontsize=7)

patches = [mpatches.Patch(color=COLORS[c], label=c) for c in class_names]
ax_demo.legend(handles=patches, loc='upper right',
               facecolor='#1a1d2e', labelcolor='#cdd6f4', fontsize=8, framealpha=0.8)

fig.suptitle('Lab 4 - LSTM Text Sentiment Analysis (PyTorch)',
             color='#cdd6f4', fontsize=14, fontweight='bold', y=0.98)

output_path = 'lab4_lstm_results.png'
plt.savefig(output_path, dpi=150, bbox_inches='tight', facecolor=fig.get_facecolor())
plt.close()
print(f'\nPlot saved to: {output_path}')

```

Результат виконання:

		Риженко Я.В			ДУ «Житомирська політехніка».26.121.20.000 – Пр4	Арк.
		Годлевський Ю.О				
Змн.	Арк.	№ докум.	Підпис	Дата		6

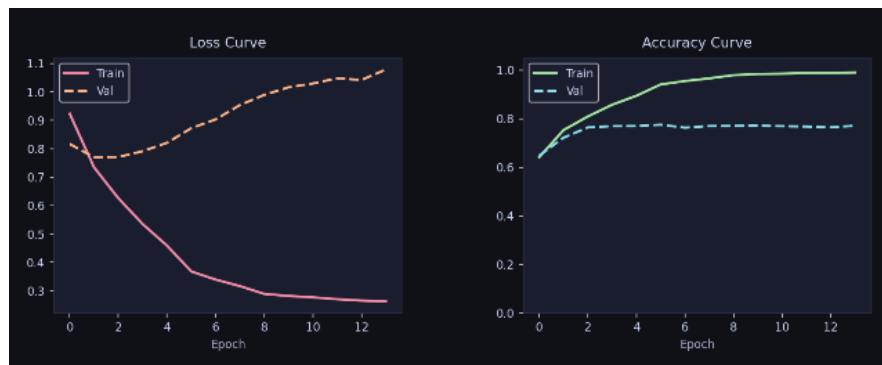


Рис. 1. Графіки навчання моделі (Loss Curve та Accuracy Curve).

Confusion Matrix				
Actual		Predicted		
		Neg	Neu	Pos
	Neg	1470	209	76
	Neu	147	366	80
Pos	69	80	303	
		Neg	Neu	Pos

Рис. 2. Матриця помилок (Confusion Matrix) для трьох класів.

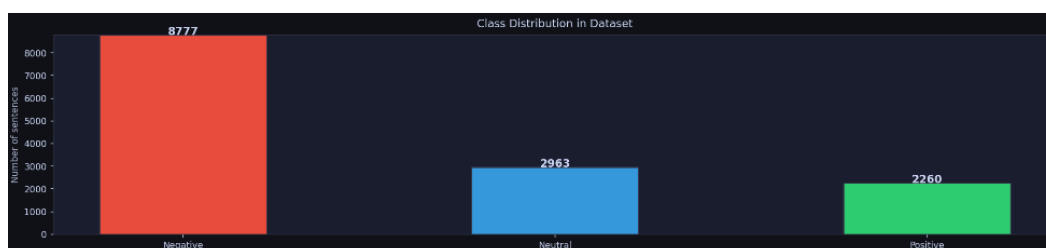


Рис. 3. Розподіл класів у датасеті.



Рис. 4. Приклади класифікації нових речень із ймовірностями класів.

Console output:

Using device: cpu

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

Dataset: osanseviro/twitter-airline-sentiment

Samples: 14000 (positive=2260, neutral=2963, negative=8777)

Vocabulary size : 5000 words

Example sentence: @SouthwestAir I got a flight at 11:55am on Thursday but looking for something tomorrow, anything available?

Token sequence : [17, 4, 94, 5, 9, 29, 75, 10, 1412, 33, 316, 8, 401, 132, 279, 322]

After padding : [17 4 94 5 9 29 75 10 1412 33 316 8 401 132

		Рижченко Я.В			ДУ «Житомирська політехніка».26.121.20.000 – Пр4	Арк.
		Годлевський Ю.О				7
Змн.	Арк.	№ докум.	Підпис	Дата		

```

279 322  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0]
Training samples: 11200 | Test samples: 2800
Model architecture:
LSTMClassifier(
  (embedding): Embedding(5000, 128, padding_idx=0)
  (lstm): LSTM(128, 96, num_layers=2, batch_first=True, dropout=0.35, bidirectional=True)
  (dropout): Dropout(p=0.35, inplace=False)
  (fc1): Linear(in_features=192, out_features=64, bias=True)
  (relu): ReLU()
  (fc2): Linear(in_features=64, out_features=3, bias=True)
)
Trainable parameters: 1,048,835
Class weights (train split): [0.5320000052452087, 1.5759999752044678, 2.065000057220459]
Epoch 1/35 loss=0.9229 acc=0.641 val_loss=0.8166 val_acc=0.648 val_f1=0.633
Epoch 5/35 loss=0.4591 acc=0.894 val_loss=0.8206 val_acc=0.770 val_f1=0.713
Epoch 10/35 loss=0.2822 acc=0.984 val_loss=1.0157 val_acc=0.771 val_f1=0.708
Early stopping at epoch 14 (best val_macro_f1=0.724)

Test accuracy: 76.4% | Loss: 0.9159

Detailed classification report:
      precision    recall  f1-score   support

Negative      0.87      0.84      0.85      1755
Neutral       0.56      0.62      0.59       593
Positive      0.66      0.67      0.67       452
accuracy                0.76      2800
macro avg      0.70      0.71      0.70      2800
weighted avg    0.77      0.76      0.77      2800
[Positive] I love this, it is truly wonderful!
[Positive] The item was delivered on Wednesday.
[Negative] This is the worst thing I have ever bought.
[Positive] Absolutely fantastic results, very happy!
[Positive] The document has been processed.
[Negative] Terrible quality, I want my money back.

Plot saved to: lab4_lstm_results.png
Process finished with exit code 0

```

Посилання на репозиторій: <https://github.com/JanRizhenko/Neural-Networks>

Висновок: У ході лабораторної роботи реалізовано задачу багатокласової класифікації тональності англомовних текстів за допомогою LSTM-моделі в PyTorch. Дані було отримано з Hugging Face, підготовлено (токенізація, словник, кодування, padding), виконано поділ на train/val/test та проведено навчання з регуляризацією і ранньою зупинкою. Отримано стабільний baseline-рівень якості (приблизно 75-76% ассигасу), при цьому найскладнішим залишається нейтральний клас. Результати підтверджують працездатність підходу й окреслюють напрями подальшого покращення (pretrained embeddings або Transformer-моделі).

		Рижченко Я.В.			ДУ «Житомирська політехніка».26.121.20.000 – Пр4	Арк.
		Годлевський Ю.О.				
Змн.	Арк.	№ докум.	Підпис	Дата		8